Imagine that temp.users_intermediate had millions of rows. You want to select rows as fast as possible.

Add a Multicolumn index covering both columns:

```
CREATE INDEX temp_users_user_id_city_id
ON temp.users_intermediate (user_id, city_id);
```

Since you have few rows in the table, PostgreSQL will choose a Sequential Scan. You want to confirm the index is posted, so disable Sequential Scans for your session by running SET enable_seqscan = OFF;.

From psql, run the following query and add EXPLAIN in front of it. Confirm the index is named and an Index Scan is used.

```
UPDATE users
SET
  city_id = temp.users_intermediate.city_id
FROM temp.users_intermediate
WHERE users.id = temp.users_intermediate.user_id
AND users.id > 0
AND users.id < 10000;
```

For your batch size, use a range that is sized appropriately for your database server, and iterate through the users table using the primary key id column. There may be deleted rows in the range, but you can plan on there being up to 10,000 rows updated at once for this batch size.

Use the PL/pgSQL Procedure that you've already learned earlier to turn this single statement into a loop.

Besides batched updates, consider adding pauses within your updates to add time for index writes and replication. Monitor server resource consumption while your statements are running.

Once the destination side table is updated, you can safely remove the temp.users table.

Wrapping Up

In this chapter you considered how to safely make database structural changes on large and busy databases. You saw lock types and queues, and learned how queries can become blocked. You simulated lock behavior and saw how to set a lock timeout and statement timeout to add more protections to your database